

```

package Assignment1;
import java.util.Random;
/**
 * CS3310 Programming Assignment #1
 */
public class Main {
    // Holds the 3 required outputs for each algorithm
    static class Result {
        final int maxSum;
        final int leftIndex; // Start boundary
        final int rightIndex; // End boundary
        Result(int maxSum, int leftIndex, int rightIndex) {
            this.maxSum = maxSum;
            this.leftIndex = leftIndex;
            this.rightIndex = rightIndex;
        }
    }
    public static void main(String[] args) {
        // Test array generation
        int n = 100;
        int[] test1 = generateRandomArray(n, -50, 50, 12345L);
        int[] test2 = generateRandomArray(n, -50, 50, 67890L);
        // Runs all 4 algorithms on BOTH test arrays and print:
        runAllAlgorithmsOnOneTest("Test Data #1", test1);
        runAllAlgorithmsOnOneTest("Test Data #2", test2);
    }
    private static void runAllAlgorithmsOnOneTest(String label, int[] a) {

        System.out.println("=====");
        ;
        System.out.println(label + " (n=" + a.length + ")");

        System.out.println("=====");
        ;
        // -----
        // Algorithm #1 (Exhaustive)
        // -----
        long start1 = System.nanoTime();
        Result r1 = maxSubSum1(a);
        long end1 = System.nanoTime();
    }
}

```

```

long cpu1 = end1 - start1;
// -----
// Algorithm #2 (Quadratic)
// -----
long start2 = System.nanoTime();
Result r2 = maxSubSum2(a);
long end2 = System.nanoTime();
long cpu2 = end2 - start2;
// -----
// Algorithm #3 (Divide & Conquer)
// -----
long start3 = System.nanoTime();
Result r3 = maxSubSum3(a);
long end3 = System.nanoTime();
long cpu3 = end3 - start3;
// -----
// Algorithm #4 (Dynamic Programming)
// -----
long start4 = System.nanoTime();
Result r4 = maxSubSum4(a);
long end4 = System.nanoTime();
long cpu4 = end4 - start4;
// -----
// Main prints results (algorithms must NOT print based off instructions)
// -----
printAlgoResult("Algorithm #1", r1, cpu1);
printAlgoResult("Algorithm #2", r2, cpu2);
printAlgoResult("Algorithm #3", r3, cpu3);
printAlgoResult("Algorithm #4", r4, cpu4);
System.out.println();
}
private static void printAlgoResult(String algoName, Result r, long cpuTimeNs) {
    System.out.println(algoName + ":");
    System.out.println("  maxSum    = " + r.maxSum);
    System.out.println("  leftIndex = " + r.leftIndex);
    System.out.println("  rightIndex = " + r.rightIndex);
    System.out.println("  CPU time (ns) = " + cpuTimeNs);
}
// =====
// TODO: Algorithm #1 (Exhaustive) O(n^3)

```

```
// =====
private static Result maxSubSum1(int[] a) {
    int n = a.length;
    int maxSum = Integer.MIN_VALUE;
    int bestLeft = 0;
    int bestRight = 0;

    for (int i = 0; i < n; i++) {
        for (int j = i; j < n; j++) {
            int thisSum = 0;
            for (int k = i; k <= j; k++) {
                thisSum += a[k];
            }
            if (thisSum > maxSum) {
                maxSum = thisSum;
                bestLeft = i;
                bestRight = j;
            }
        }
    }
    return new Result(maxSum, bestLeft, bestRight);
}

// =====
// TODO: Algorithm #2 (Quadratic) O(n^2)
// =====
private static Result maxSubSum2(int[] a) {
    int n = a.length;
    int maxSum = Integer.MIN_VALUE;
    int bestLeft = 0;
    int bestRight = 0;

    for (int i = 0; i < n; i++) {
        int thisSum = 0;
        for (int j = i; j < n; j++) {
            thisSum += a[j];
            if (thisSum > maxSum) {
                maxSum = thisSum;
                bestLeft = i;
                bestRight = j;
            }
        }
    }
}
```

```

    }
    }
    return new Result(maxSum, bestLeft, bestRight);
}
// =====
// TODO: Algorithm #3 (Divide & Conquer) O(n log n)
// =====
private static Result maxSubSum3(int[] a) {
    // Just to check if "hypothetically" theres nothing in the array
    if (a.length == 0) {
        return new Result(0, -1, -1);
    }
    return maxSumRec(a, 0, a.length - 1);
}
private static Result maxSumRec(int[] a, int left, int right) {
    if (left == right) {
        return new Result(a[left], left, right);
    }

    int center = (left + right) / 2;

    Result leftRes = maxSumRec(a, left, center);
    Result rightRes = maxSumRec(a, center + 1, right);

    // Best suffix ending at center
    int leftBorderSum = 0;
    int maxLeftBorderSum = Integer.MIN_VALUE;
    int crossLeftStart = center;

    // Going down left side from center
    for (int i = center; i >= left; i--) {
        leftBorderSum += a[i];
        if (leftBorderSum > maxLeftBorderSum) {
            maxLeftBorderSum = leftBorderSum;
            crossLeftStart = i;
        }
    }

    // Best prefix starting at center+1
    int rightBorderSum = 0;

```

```

int maxRightBorderSum = Integer.MIN_VALUE;
int crossRightEnd = center + 1;

// Going up right from center + 1
for (int j = center + 1; j <= right; j++) {
    rightBorderSum += a[j];
    if (rightBorderSum > maxRightBorderSum) {
        maxRightBorderSum = rightBorderSum;
        crossRightEnd = j;
    }
}

Result crossRes = new Result(maxLeftBorderSum + maxRightBorderSum,
crossLeftStart, crossRightEnd);

return bestOfThree(leftRes, rightRes, crossRes);
}

private static Result bestOfThree(Result a, Result b, Result c) {
    Result best = a;
    if (b.maxSum > best.maxSum) best = b;
    if (c.maxSum > best.maxSum) best = c;
    return best;
}

// =====
// Algorithm #4 (Dynamic Programming) O(n)
// =====

private static Result maxSubSum4(int[] a) {
    int n = a.length;
    if (n == 0) {
        return new Result(0, -1, -1);
    }

    int bestSum = a[0];
    int bestLeft = 0;
    int bestRight = 0;

    int currentSum = a[0];
    int currentLeft = 0;

```

```

        for (int i = 1; i < n; i++) {
            // Decide, extend previous subarray or start fresh at i
            if (currentSum + a[i] < a[i]) {
                currentSum = a[i];
                currentLeft = i;
            } else {
                currentSum = currentSum + a[i];
            }

            // Update global best
            if (currentSum > bestSum) {
                bestSum = currentSum;
                bestLeft = currentLeft;
                bestRight = i;
            }
        }

        return new Result(bestSum, bestLeft, bestRight);
    }

    // =====
    // Helper this is for data generation
    // =====
    private static int[] generateRandomArray(int n, int minInclusive, int maxInclusive, long
seed) {
        Random rng = new Random(seed);
        int[] a = new int[n];
        int bound = (maxInclusive - minInclusive) + 1;
        for (int i = 0; i < n; i++) {
            a[i] = rng.nextInt(bound) + minInclusive;
        }
        return a;
    }
}

```

terminated: main (1) [Java Application], C:\Users\amara\AppData\Local\Programs\OpenJDK\bin\java.exe

```
=====
Test Data #1 (n=100)
=====
Algorithm #1:
    maxSum      = 496
    leftIndex   = 4
    rightIndex  = 57
    CPU time (ns) = 5315700
Algorithm #2:
    maxSum      = 496
    leftIndex   = 4
    rightIndex  = 57
    CPU time (ns) = 121100
Algorithm #3:
    maxSum      = 496
    leftIndex   = 4
    rightIndex  = 57
    CPU time (ns) = 131000
Algorithm #4:
    maxSum      = 496
    leftIndex   = 4
    rightIndex  = 57
    CPU time (ns) = 10100
```

```
=====
Test Data #2 (n=100)
=====
Algorithm #1:
    maxSum      = 291
    leftIndex   = 0
    rightIndex  = 49
    CPU time (ns) = 4104500
Algorithm #2:
    maxSum      = 291
    leftIndex   = 0
    rightIndex  = 49
    CPU time (ns) = 160400
Algorithm #3:
    maxSum      = 291
    leftIndex   = 0
    rightIndex  = 49
    CPU time (ns) = 143900
Algorithm #4:
    maxSum      = 291
    leftIndex   = 0
    rightIndex  = 49
    CPU time (ns) = 10500
```